

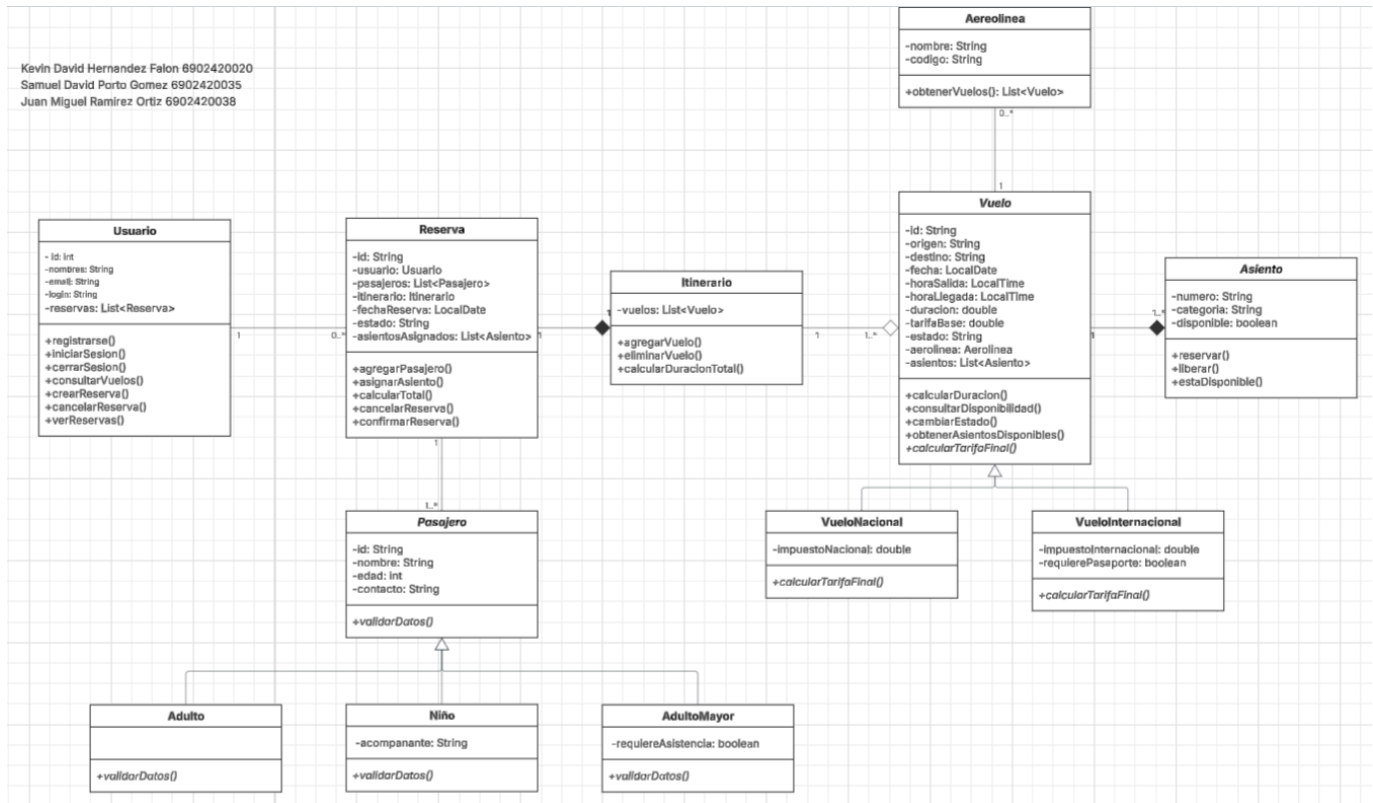
Entregable 1: Diagrama de clases UML

Kevin David Hernandez Falon 6902420020

Samuel David porto Gomez 6902420035

Juan Miguel Ramirez Ortiz: 6902420038

1. Primera versión del diagrama de clases:



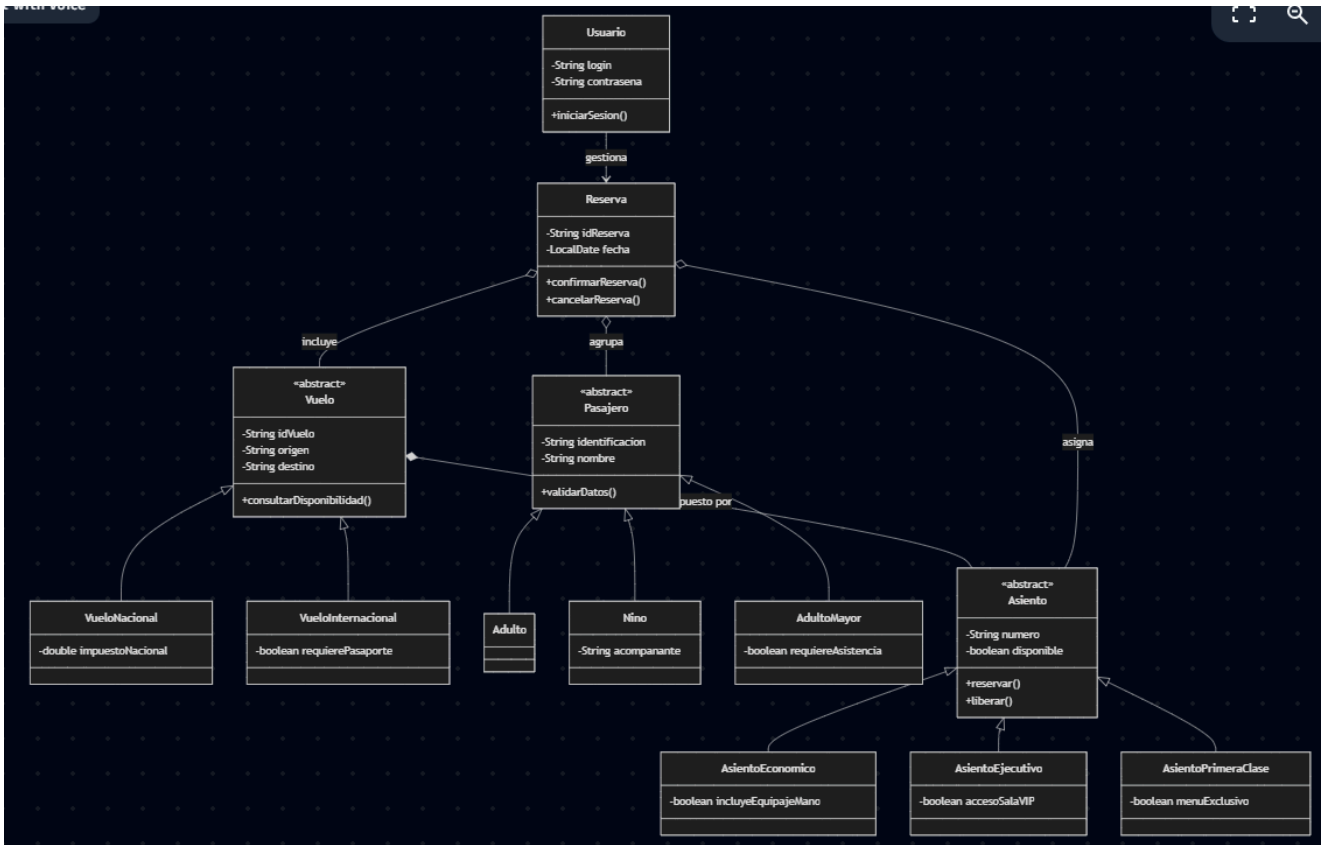
En esta primera versión del diagrama de clases y análisis del enunciado, identificamos las clases principales del problema (Usuario, Reserva, Itinerario, Vuelo, Asiento, Pasajero y Aerolínea). implementamos herencia en Vuelo (Nacional e Internacional) y Pasajero (Adulto, Niño, AdultoMayor). Sin embargo, mantuvimos estructuras más simples en otros componentes: modelamos Asiento como una clase única donde la categoría era solo un atributo de tipo *String*, controlamos la asignación de sillas en la clase Reserva mediante un atributo de tipo entero (int asientosAsignados), y establecimos una relación de agregación entre Itinerario y Vuelo.

2. Prompt utilizado para solicitar la revisión o mejora del diseño a la IA:

"Actúa como un arquitecto de software experto en Java y POO. Tenemos un diagrama inicial para un sistema de reservas de vuelos. Actualmente usamos un atributo 'int asientosAsignados' en Reserva, tenemos una sola clase Asiento con un String para la categoría, y usamos agregación(rombo blanco) entre Itinerario y Vuelo. ¿Qué mejoras

nos sugieres aplicando principios de POO como herencia, polimorfismo y validación de relaciones?"

3. Respuesta generada por la IA:



La IA sugirió varios cambios en el diagrama:

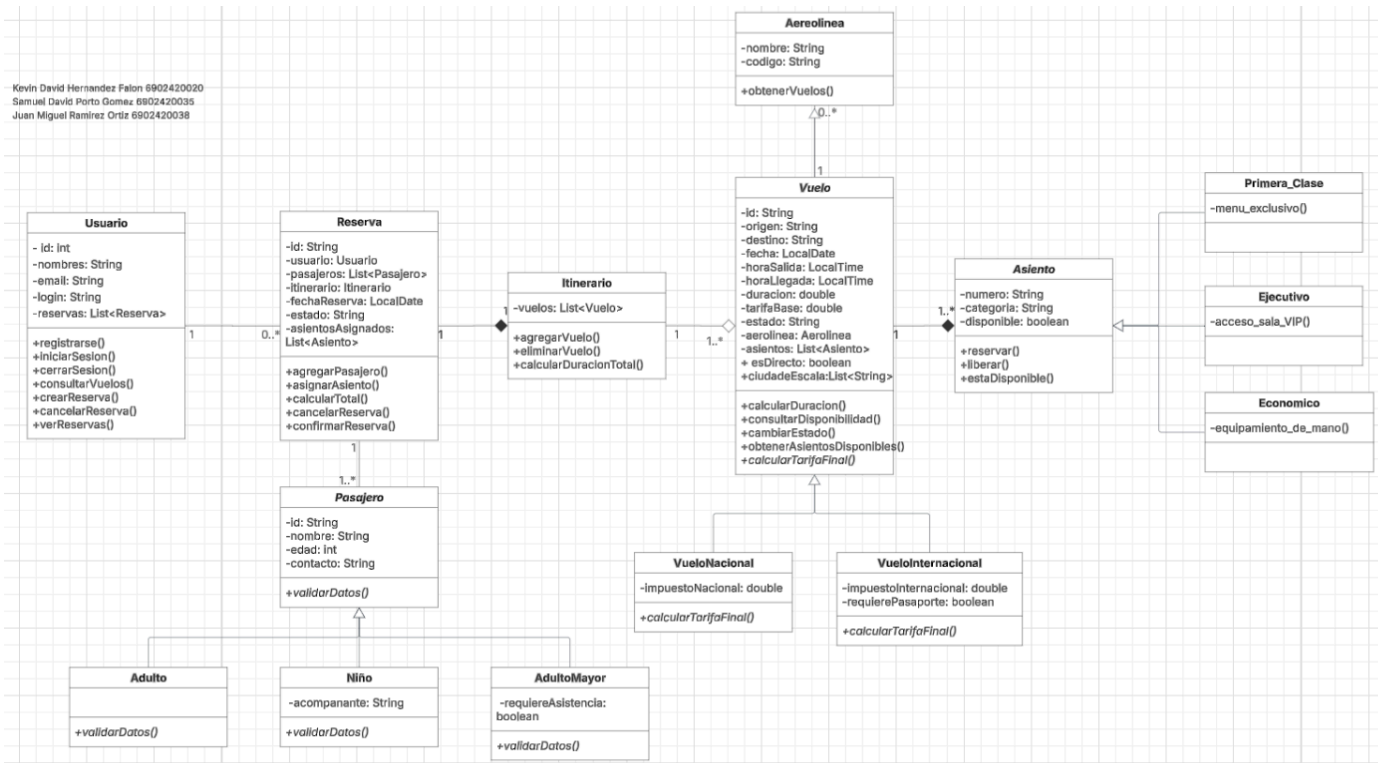
- **Herencia en Asientos:** Crear una clase base Asiento y subclases para las categorías (Primera_Clase, Ejecutivo, Economico) para manejar atributos específicos.
- **Gestión de Asientos:** Cambiar el contador entero de asientos en Reserva por una lista de objetos List<Asiento> para evitar sobreventas.
- **Encapsulamiento en Asiento:** Agregar métodos de validación de estado y atributos detallados para que el asiento gestione su propia disponibilidad.

Elemento analizado	Diseño inicial del grupo	Sugerencia de la IA	Decisión final del grupo	Justificación
Categorías de Asientos	Clase única Asiento con un atributo String categoria.	Crear subclases (Economico, Ejecutivo, Primera_Clase) heredando de Asiento.	Se analizó la sugerencia y se agregó la herencia al diagrama.	Permite asignar atributos únicos a cada categoría (ej. menu_exclusivo o acceso_sala_VIP) y

				facilita el polimorfismo.
Asignación de Asientos	Atributo asientosAsignados: int en la clase Reserva.	Cambiar a una lista de objetos: List<Asiento>.	Se analizó la sugerencia y luego se actualizó el atributo en la clase Reserva.	Un número entero no identifica qué silla se reservó. La lista de objetos garantiza que no se asigne el mismo asiento dos veces.
Relación Reserva-itinerario	Asociación simple	Cambiar a composición(una reserva contiene sólo un itinerario)	Se implementó composición	Una reserva "es dueña" de su itinerario. Tienen una relación de ciclo de vida dependiente: si la reserva se elimina o cancela por completo, ese itinerario específico armado por el usuario pierde su propósito y también deja de existir.
Clase pasajero	Clase única	Usar herencia(Adulto, NIÑO, Adulto Mayor)	Se implementó Herencia	Permite aplicar polimorfismo y separar responsabilidades. Evita tener atributos nulos en una clase gigante y permite que cada tipo de pasajero tenga sus propias reglas o datos (ej. un niño necesita el dato de un acompañante, un adulto mayor requiere asistencia).
Clase Asiento	Solo número y estado	Agregar categoría(económica, ejecutiva)	Se añadieron los atributos categoría, disponible y métodos de validación.	Mejora el encapsulamiento y el modelado del dominio. Un asiento debe ser capaz de gestionar su propio estado de ocupación a través de métodos propios (reservar(), liberar()), garantizando que la disponibilidad se actualice de forma segura.
Relación Vuelo-Asiento	Asociación	Composición	Se implementó composición entre Vuelo y Asiento	Representa una relación física de "todo-parte" fuerte. Un asiento no tiene sentido ni existencia

				lógica en este sistema fuera del vuelo al que pertenece; si el vuelo se borra, sus asientos también.
--	--	--	--	--

5. Diagrama de clases UML final:



6. Explicación breve de las decisiones de diseño: El diseño final se estructuró buscando la máxima modularidad y reutilización de código. Se aplicó herencia en tres frentes fundamentales: Pasajero (Adulto, Niño, AdultoMayor), Vuelo (Nacional, Internacional) y Asiento (Económico, Ejecutivo, Primera Clase), lo cual permite tratar a todos los objetos genéricos polimórficamente, mientras se mantienen sus comportamientos particulares.

Para fortalecer esta estructura, se decidió definir las clases *Vuelo*, *Pasajero* y *Asiento* como **clases abstractas** Esta decisión garantiza que el sistema no instancie conceptos

generales, sino obligatoriamente objetos específicos (por ejemplo, impidiendo crear un 'Pasajero' genérico y obligando a instanciar un *Adulto*, *Niño* o *AdultoMayor*).

Adicionalmente, se definieron relaciones de composición (Vuelo - Asiento, ya que los asientos son parte física del avión) y agregación (Reserva - Pasajeros, ya que el usuario sigue existiendo en el sistema aunque se cancele la reserva), cumpliendo fielmente con los requerimientos del dominio del problema.